could potentially take on highly varying values, causing issues when ingesting and storing the metrics (costs) as well as when querying them. In the context of queries, high cardinality might cause slow or hanging queries—that is, a query might time out before returning a result.

The best way to address this problem is to avoid high cardinality when using metrics backends. By avoiding it, I mean carefully controlling which metrics you scrape (or collect) and working backward from what you actually use. For example, your guidance could be that only metrics that are actively used in a dashboard or an alert are "allowed in." You can defer the decision by implementing this logic in the agent—for example, via an OpenTelemetry processor in one of your pipelines.

Alternatively, if you can't or don't want to limit yourself, you can use dedicated data stores that can handle a high volume of metrics, such as ClickHouse or commercial offerings. For example, Honeycomb initially spent a considerable time to build a proprietary data store for high-cardinality signals, from scratch. In their case, it made sense, since that's literally their business model. Again, this is a quick reminder of buy versus build in this context.

In general, query performance (how fast a backend can return results as a reaction to a query) is an important topic; however, details are beyond the scope of this chapter. It does have a time–speed tradeoff in some systems; for example, Prometheus and InfluxDB are fast performers if you're within their cardinality limits and are doing "alerting"- style queries. Distributed systems, on the other hand, tend to be slower in the near term but handle long-range queries (i.e., "tell me how we performed for the last quarter") far better.

If you're interested in setting up your own query performance TSDB benchmark, have a look, for example, at what Aliaksandr Valialkin shared in "High-Cardinality TSDB Benchmarks: VictoriaMetrics vs. TimescaleDB vs. InfluxDB" (http://mng.bz/D4mn).

Concerning the way queries are expressed as well as wire formats, some commonly used query languages, schemas, and formats are as follows (note that I intentionally mix up transport schema and query languages here to keep things simple):

- For the logs signal type:
  - Elastic Common Schema (ECS; http://mng.bz/lWy6) with the query syntax of Elastic-flavored Lucene
  - Grafana Labs' LogQL (https://grafana.com/docs/loki/latest/logql/)
  - CloudWatch Logs Insights query syntax (http://mng.bz/BmJ0)
  - The syslog protocol (https://datatracker.ietf.org/doc/rfc5424/), as per RFC 5424
  - Apache's Common Log Format (https://en.wikipedia.org/wiki/Common_Log_Format)
  - NGINX access and error log (https://docs.nginx.com/nginx/admin-guide/monitoring/logging)